

JVM & Packages & Collections

JDK, JRE, JVM

JVM – Java virtual machine, виртуальная машина, в которой запускаются программы, написанные на Java

JRE – Java runtime environment, среда выполнения для Java, содержит библиотеки классов, загрузчик классов, JVM

JDK – Java development kit, средства, позволяющие разрабатывать на Java. Основным на текущий момент является OpenJDK

Спецификация JVM

Первая часть JVM — **спецификация**, которая до конца не определяет все детали реализации виртуальной машины. Это значит, что остается максимальная свобода творчества для разработчика, который работает с ней. Чтобы правильно реализовать виртуальную машину Java, вам нужно всего лишь уметь читать class-файлы и правильно выполнять указанные в них операции.

Итак, все, что должна делать JVM — правильно запускать Java-программы. Это может показаться достаточно простым процессом, однако это очень масштабная задача, учитывая мощность и гибкость языка Java.

Реализация JVM

Реализация спецификации JVM приводит к созданию реальной программы, которая и является реализацией JVM. По сути, существует огромное количество реализаций спецификации JVM — как коммерческих, так и с открытым кодом.

Экземпляр JVM

После того, как спецификация JVM реализована и выпущена в качестве самостоятельной программы, вы можете загрузить ее как приложение. Эта загруженная программа является экземпляром виртуальной машины.

Чаще всего, когда разработчики говорят о JVM, они имеют ввиду экземпляр JVM, который работает в среде разработки.

JVMs

«HotSpot» — это основная виртуальная машина Java (JVM) как для клиентских, так и для серверных компьютеров, выпускаемая корпорацией «Oracle».

https://en.wikipedia.org/wiki/List_of_Java_virtual_machines

Версия	Дата выпуска	Конец бесплатной поддержки	Конец LTS
JDK Beta	1995	неизвестно	неизвестно
JDK 1.0	январь 1996		
JDK 1.1	февраль 1997		
J2SE 1.2	декабрь 1998		
J2SE 1.3	май 2000		
J2SE 1.4	февраль 2002	октябрь 2008	февраль 2013
J2SE 5.0	сентябрь 2004	ноябрь 2009	апрель 2015
Java SE 6	декабрь 2006	апрель 2013	декабрь 2018
Java SE 7	июль 2011	апрель 2015	июль 2022
Java SE 8 (LTS)	март 2014	- Январь 2019 (коммерческая лицензия Oracle) - Декабрь 2020 (лицензия для персонального использования Oracle) - Сентябрь 2023 (для AdoptOpenJDK^[1])	март 2025
Java SE 9 ^{[2][3][4]}	сентябрь 2017	Март 2018 (OpenJDK)	–
Java SE 10	март 2018	Сентябрь 2018 (OpenJDK)	–
Java SE 11 (LTS)	сентябрь 2018	Сентябрь 2022 ^[1]	сентябрь 2026
Java SE 12	март 2019	Сентябрь 2019	–
Java SE 13	сентябрь 2019	Март 2020	–
Java SE 14	март 2020	Сентябрь 2020	
Java SE 15	сентябрь 2020	Март 2021	
Java SE 16	март 2021	Сентябрь 2021 (OpenJDK)	
Java SE 17 (LTS)	сентябрь 2021	Сентябрь 2030	

Java SE 18		62	22nd March 2022	September 2022	—
Java SE 19		63	20th September 2022	March 2023	—
Java SE 20		64	21st March 2023	September 2023	—
Java SE 21	LTS	65	19th September 2023	September 2028 for Oracle ^[4] September 2028 for Microsoft Build of OpenJDK ^[18] December 2029 for Red Hat ^[10] December 2029 for Eclipse Temurin ^[15] October 2030 for Amazon Corretto ^[16] September 2031 for Azul ^[9] December 2029 for IBM Semeru Runtimes ^[17]	September 2031 for Oracle ^[4] March 2032 for BellSoft Liberica ^[12]
Java SE 22		66	19th March 2024	September 2024	—
Java SE 23		67	17th September 2024	March 2025 for Oracle September 2032 for Azul ^[9] March 2025 for IBM Semeru Runtimes ^[17]	—
Java SE 24		68	March 2025	September 2025	—
Java SE 25	LTS	69	September 2025	September 2030 for Oracle ^[4]	September 2033 for Oracle ^[4] March 2034 for BellSoft Liberica ^[12]
Legend: Old version, not maintained Old version, still maintained Latest version Future release					

Java Editions

- **Java SE (Java Platform Standard Edition)** - это стандартная редакция Java, которая используется для разработки простых Java приложений. Например, можно создавать консольные приложения, апплеты, приложения с графическим интерфейсом.
- **Java EE (Java Platform Enterprise Edition)** - это редакция Java для разработки распределенных приложений масштаба предприятий. Включает в себя технологии EJB, JPA, Servlets, JMS и др.
- **Java ME (Java Platform Micro Edition)** - это редакция Java для разработки приложений для микрокомпьютеров, мобильных платформ и т.д.

А кто и как формирует спецификации?

Как?

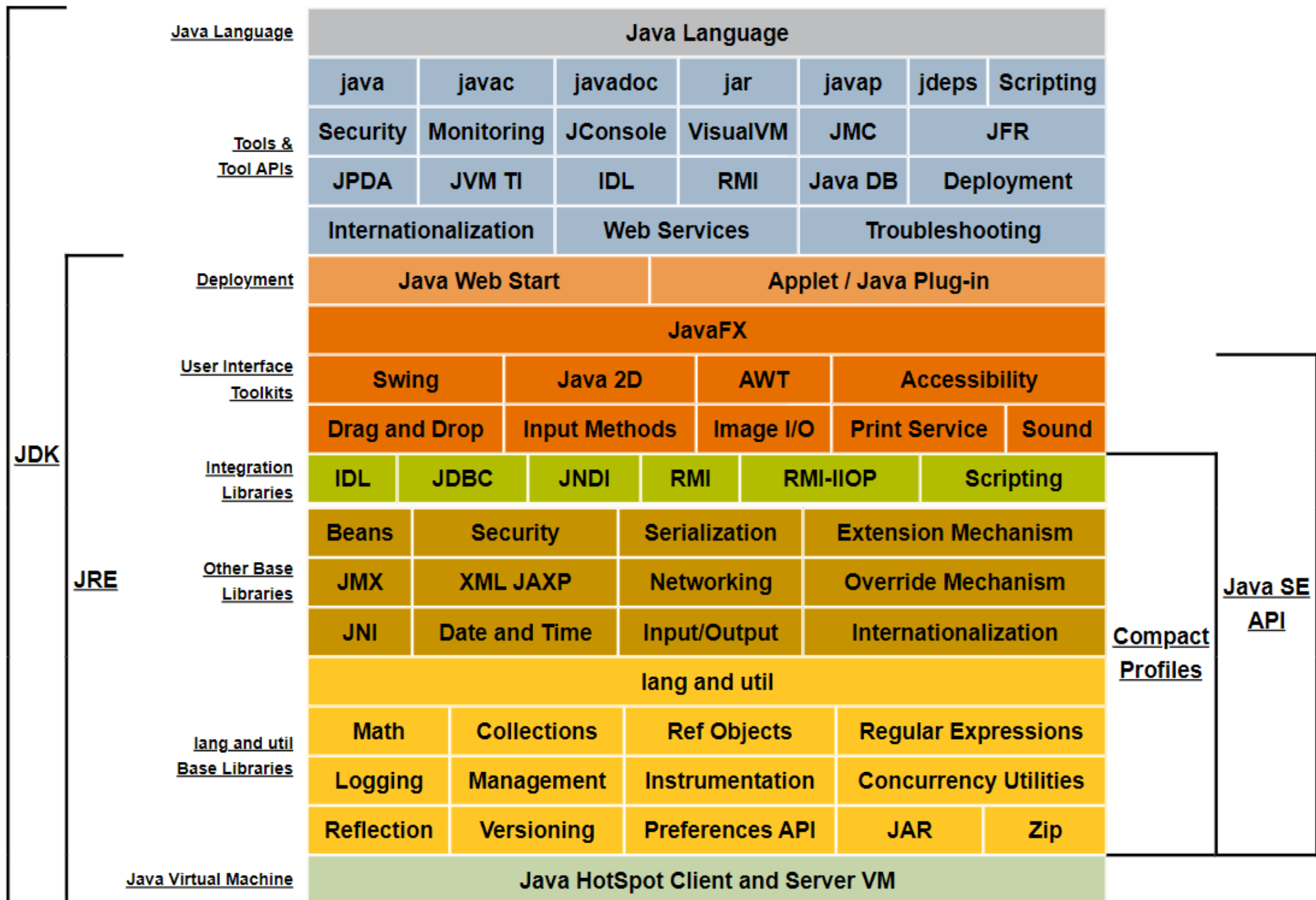
- [https://ru.wikipedia.org/wiki/Java Community Process](https://ru.wikipedia.org/wiki/Java_Community_Process)

Кто?

- <https://jcp.org/ja/participation/committee>

Процесс в деталях:

- <https://www.jcp.org/aboutJava/communityprocess/ec-public/materials/2023-06-13/Contributing-to-OpenJDK-2023-06.pdf>



Packages

Как правило, в Java классы объединяются в пакеты. Пакеты позволяют организовать классы логически в наборы. По умолчанию java уже имеет ряд встроенных пакетов, например, **java.lang**, **java.util**, **java.io** и т.д. Кроме того, пакеты могут иметь вложенные пакеты.

Организация классов в виде пакетов позволяет избежать конфликта имен между классами. Ведь нередки ситуации, когда разработчики называют свои классы одинаковыми именами. Принадлежность к пакету позволяет гарантировать однозначность имен.

Packages

Классы необязательно определять в пакеты. Если для класса пакет не определен, то считается, что данный класс находится в пакете по умолчанию, который не имеет имени.

Если нам надо использовать классы из других пакетов, то нам надо подключить эти пакеты и классы. Исключение составляют классы из пакета **java.lang** (например, `String`), которые подключаются в программу автоматически.

Java Collection Framework

- Collection Framework — иерархия интерфейсов и их реализаций, которая является частью JDK и позволяет разработчику пользоваться большим количеством структур данных с которыми вы привыкли работать сразу из «коробки».
- Соответственно, для описание такого общего поведения для всех коллекций написали разработчики Java интерфейс **java.util.Collection**
- <https://docs.oracle.com/javase/8/docs/api/java/util/Collection.html>

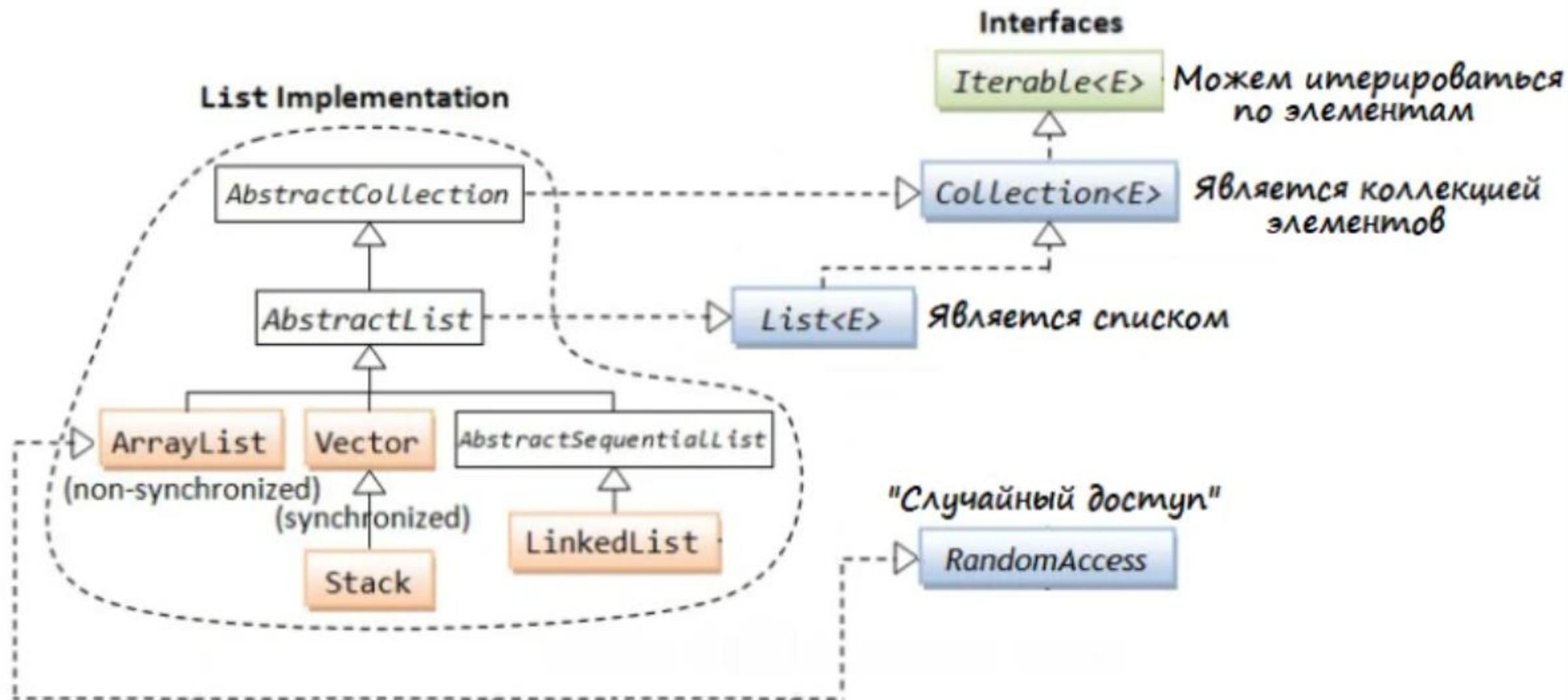
Java Collection Framework

- Интерфейс **Collection** — это то место, откуда берут начало все коллекции. **Collection** — это идея, это представление о том, как должны себя вести все коллекции. Поэтому, термин "Коллекция" выражена в виде интерфейса. Естественно, интерфейсу нужны реализации.
- Интерфейс **java.util.Collection** имеет абстрактный класс **AbstractCollection**, то есть некоторая "абстрактная коллекция", которая представляет собой скелет для остальных реализаций (о чём написано [в JavaDoc](#) над классом **java.util.AbstractCollection**).

Java Collection Framework

Вспомним, что в шарпах, если мы хотим итерироваться по коллекциям, то нам нужен был любой объект который может предоставить метод для получения итератора. Это очень важная концепция соблюдается и в Java. Поэтому, интерфейс **Collection** наследуется от **Iterable**, т.к. во-первых, всё что **Iterable** должно уметь возвращать **Iterator** по своему содержимому. А во-вторых, всё что **Iterable** может использоваться в циклах for-each-loop. И именно при помощи итератора в **AbstractCollection** реализованы такие методы, как contains, toArray, remove. И путь к познанию коллекций начинается с одной из самых распространённых структур данных — списка, т.е. **List**.

List Implementation



java.util.List

Среди всех реализаций можно выделить две самые часто используемые: ArrayList и LinkedList. Во-первых, ArrayList — это список (List) на основе массива (Array). Это позволяет добиться "Произвольного доступа" (Random Access) к элементам.

Произвольный доступ — это возможность сразу достать элемент по индексу, а не перебирать все элементы, пока не найдём элемент с нужным индексом. Именно массив как основа позволяет этого достичь.

java.util.List

Напротив, `LinkedList` — это связанный (`Linked`) список (`List`). Каждая запись в связанном списке представлена в виде `Entry`, которая хранит сами данные, а так же ссылку на следующую (`next`) и предыдущую (`previous`) `Entry`. Таким образом `LinkedList` реализует "Последовательный доступ" (`Sequential Access`).

java.util.List

Массив, как мы знаем, не может изменять свой размер. Как же работает тогда ArrayList?

А работает он очень просто. Когда заканчивается место в массиве, то он увеличивается в 1.5 раза (примерно так же как и в шарпах аллокатор получает дополнительное место в куче).

java.util.Vector

Рассмотрев ArrayList нельзя не вспомнить про его "предшественника", про класс **java.util.Vector**.

Отличается Vector от ArrayList тем, что методы для работы с коллекцией (добавление, удаление и т.д.) **синхронизированы**. То есть если один поток (Thread) будет добавлять элементы, то другие потоки будут ждать, пока первый поток не закончит свою работу. Так как потокобезопасность зачастую не требуется, рекомендуется использовать в таких случаях класс ArrayList, о чём прямым текстом сказано в JavaDoc для класса Vector.

Потокобезопасность коллекций

- **Synchronized** коллекции обычно означают потокобезопасный вариант, т.е. только один поток параллельно может работать с коллекцией. Синхронизированные коллекции обеспечивают согласованность/целостность данных в многопоточных средах. Однако они могут привести к снижению производительности, так как только один поток может одновременно получить доступ к коллекции
- **Non-Synchronized** коллекции наоборот, не потокобезопасны, т.е. множество различных потоков может работать с коллекцией.

java.util.Vector

Кроме того, Vector увеличивает свой размер не в 1.5 раза, как ArrayList, а в 2 раза. В остальном поведение такое же — за Vector скрывается хранилище элементов в виде массива и добавление/удаление элементов имеют те же последствия, что и в ArrayList.

На самом деле, про Vector мы вспомнили не просто так. Если посмотреть в Javadoc, то мы увидим в "Direct Known Subclasses" такую структуру, как java.util.Stack.

Стэк — LIFO структура last-in-first-out (последним пришёл, первым ушёл), реализует дополнительные методы: peek, pop, push.

List Implementation

AbstractCollection

AbstractList

ArrayList

(non-synchronized)

Vector

(synchronized)

Stack

AbstractSequentialList

LinkedList

RandomAccess

"Случайный доступ"

Interfaces

Iterable<E>

Можем итерироваться по элементам

Collection<E>

Является коллекцией элементов

List<E>

Списки

Queue<E>

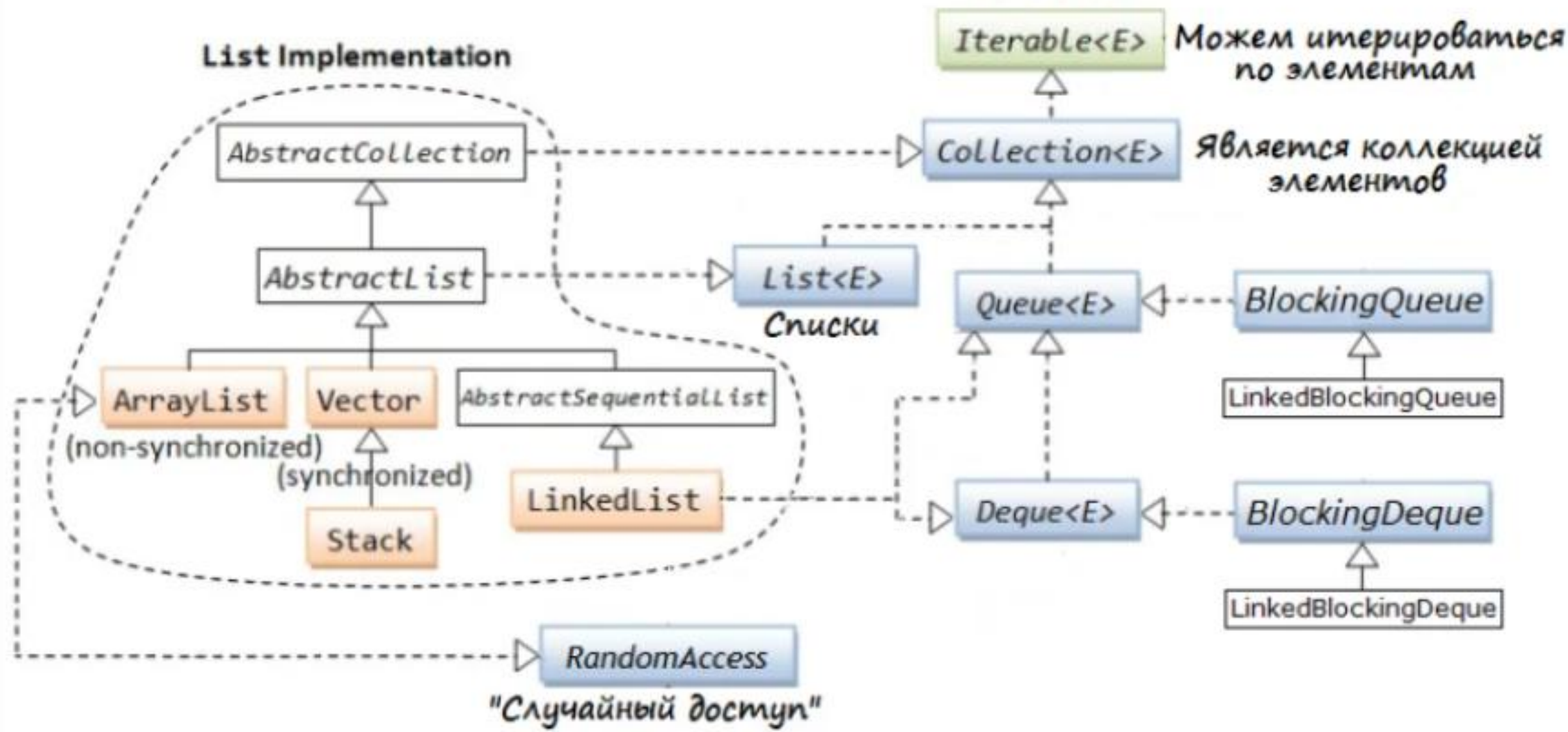
BlockingQueue

LinkedBlockingQueue

Deque<E>

BlockingDeque

LinkedBlockingDeque



java.util.Queue

- Очередь (Queue) — это структура, знакомая нам из жизни. Очереди в магазины, к врачам. Кто первое пришёл (First In), тот первое и выйдет из очереди (First Out). В Java очередь представлена интерфейсом `java.util.Queue`.
- Согласно Javadoc очереди, очередь добавляет следующие методы:

Summary of Queue methods

	<i>Throws exception</i>	<i>Returns special value</i>
Insert	<code>add(e)</code>	<code>offer(e)</code>
Remove	<code>remove()</code>	<code>poll()</code>
Examine	<code>element()</code>	<code>peek()</code>

java.util.Deque

У интерфейса очереди есть так же полезный наследник — Deque. Это так называемая "двусторонняя очередь". То есть такая очередь позволяет использовать эту структуру как с начала, так и с конца.

В качестве реализации java.util.Deque можно привести класс java.util.ArrayDeque. То есть списки можно реализовать при помощи связанного списка и массива и очереди тоже можно реализовать при помощи массива или при помощи связанного списка.

Интерфейсы Queue и Deque имеют наследников, представляющих "блокирующую очередь": BlockingQueue и BlockingDeque.

java.util.Deque

Давайте посмотрим на какие-нибудь примеры блокирующих очередей.

BlockingQueue реализуют: PriorityQueue, SynchronousQueue, ArrayBlockingQueue, DelayQueue, LinkedBlockingQueue.

А вот BlockingDeque реализуют из стандартного Collection Frameworks всего LinkedBlockingDeque. Каждая очередь может быть использована для решения различного рода задач многопоточного программирования, но сегодня это не входит в рамки нашей лекции.

java.util.Set

Лучше давайте добавим ещё одну ветвь иерархии Collection Framework'a — Set.

От очереди и списка Set отличается большей абстракцией над хранением элементов. Set — как мешок с предметами, где неизвестно, как лежат предметы и в каком порядке они легли.

В Java такой набор представлен интерфейсом `java.util.Set`.

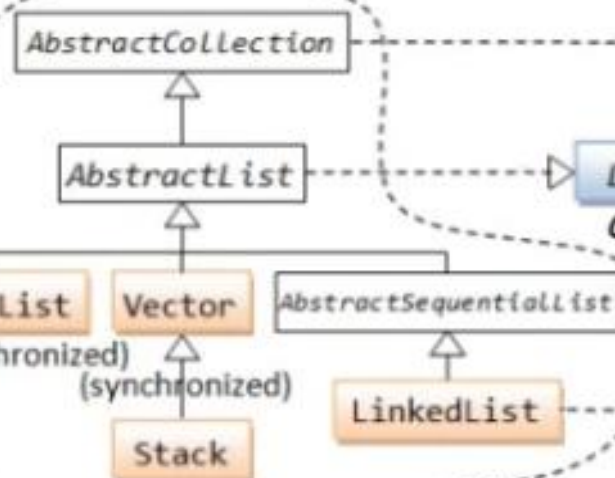
java.util.Set

Интересно, что сам интерфейс Set не добавляет новых методов к интерфейсу Collection, а лишь уточняет требования (про то, что не должно содержать дубликатов). Кроме того, из прошлого описания следует, что просто так из Set нельзя получить элемент. Для получения элементов используется Iterator.

Во-первых, самая известная реализация — на основе хэш-кода — HashSet.

Другая не менее известная реализация — на основе дерева — TreeSet.

List Implementation



RandomAccess
"Случайный доступ"

Interfaces

Iterable<E> Можем итерироваться по элементам

Collection<E> Является коллекцией элементов

List<E>
Списки

Queue<E>

BlockingQueue

LinkedBlockingQueue

Deque<E>

BlockingDeque

LinkedBlockingDeque

Set<E>

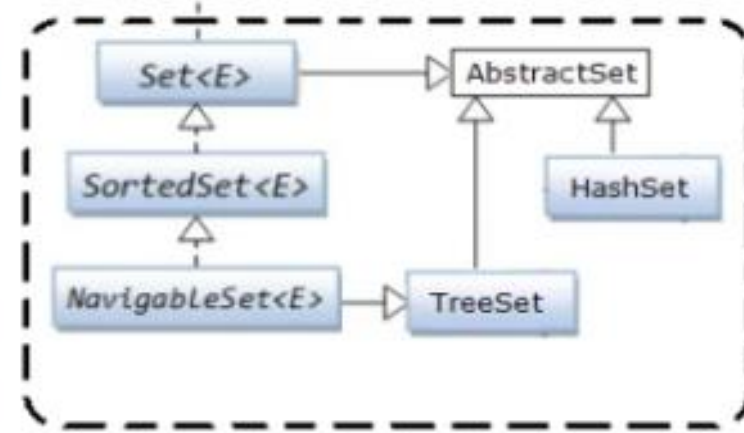
AbstractSet

SortedSet<E>

HashSet

NavigableSet<E>

TreeSet



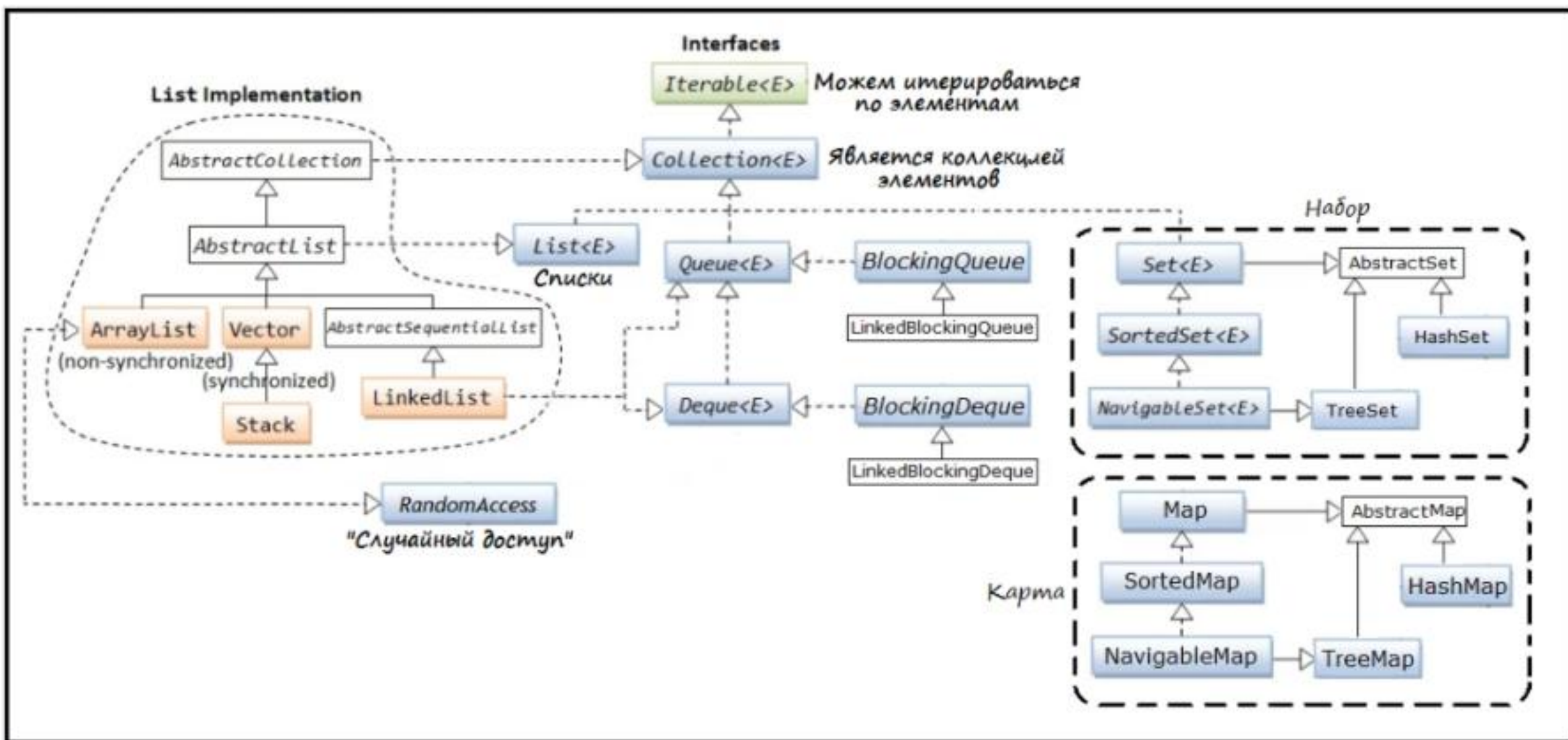
java.util.Map

- В рамках коллекций осталось разобрать последнюю иерархию, которая на первый взгляд стоит в стороне — java.util.Map.
- Карты — это такая структура данных, в которой данные хранятся по ключу. Например, ключом может служить ID или код города. Аналог из мира C# это словари.

java.util.Map

При помощи `keySet` позволяет получить набор ключей
А при помощи метода `values` можем получить коллекцию значений
в карте. Ключи в карте уникальны, что подчёркивается структурой
данных `Set`. Значения же могут повторяться, что подчёркивает
структура данных `Collection`.

Кроме того, при помощи метода `entrySet` можем получить набор
пар "ключ - значение".



Другие имплементации коллекций

При необходимости, разработчик может создать собственную реализацию, расширив или переопределив существующую логику, либо создав свою собственную реализацию подходящего интерфейса с нуля. Также существует некоторое количество готовых решений, которые являются альтернативой или дополнением к Java Collections Framework. Наиболее популярными являются **Google Guava** и **Apache Commons Collections**. (Эти пакеты выступают как **boost** для C++)

А как работать с коллекциями?

Также хочется отметить, что существуют потокобезопасные версии коллекций, прямо как в C#, лежат они в отдельном пакете `java.util.concurrent`

Остаётся лишь научиться работать с коллекциями так же как мы это с вами делали на ООП

Stream API (или LINQ в Java)

Стримы и коллекции чем-то похожи друг на друга, но у них разное назначение. Коллекции обеспечивают эффективный доступ к одиночным объектам, а стримы, наоборот, для прямого доступа и обработки отдельных элементов не используются. Стримы предназначены для параллельных и последовательных агрегаций, выполняемых через цепочку методов, прямо как LINQ.

Для того чтобы начать строить свою цепочку преобразований, первым делом необходимо получить поток, для этого, над любой коллекцией достаточно вызвать метод **.stream()**

Stream API

Так же можно создать Stream и не имея коллекции:

- Например, Пустой стрим: `Stream.empty()`
- Или из указанных элементов: `Stream.of("1", "2", "3")`

Stream API



Далее, есть такое понятие как операторы (по сути методы класса Stream)

Промежуточные (“intermediate”, ещё называют “lazy”) — обрабатывают поступающие элементы и возвращают стрим. Промежуточных операторов в цепочке обработки элементов может быть много. (Например `.Where(...)` из LINQ)

Терминальные (“terminal”, ещё называют “eager”) — обрабатывают элементы и завершают работу стрима, так что терминальный оператор в цепочке может быть только один. (Например `.ToList()` из LINQ)

Stream API

Важные моменты:

- Обработка не начнётся до тех пор, пока не будет вызван терминальный оператор.
- Экземпляр, стрима нельзя использовать более одного раза (В отличии от `IEnumerable` из C#)

Поэтому, вам необходимо будет открывать новый поток каждый раз если планируется работать с одной коллекцией по разному

Stream API

```
list
```

```
    .stream()  
    .filter(x -> x.toString().length() == 3)  
    .forEach(System.out::println);
```

```
list.stream().forEach(x -> System.out.println(x));
```


C#	Java Stream API Method
.Where(predicate)	.filter(predicate)
.Select(Func<TSource, TResult>)	.map(Function<T,R>) / .map(mapper)
.SelectMany(...)	.flatMap(...)
Enumerable.Range	IntStream.range
.Take()	.limit()
.Skip()	.skip()
.Distinct()	.distinct()
.OrderBy(...)	.sorted(comparator)
List.ForEach(...)	.forEach(Consumer action)
.Count()	.count()
.Aggregate()	.reduce()
.First()	.findFirst()
.Any()	.anyMatch()
.Any() с отрицанием	.noneMatch()
.ToList()	.toList()
.Distinct().ToArray()	.toSet()
String.Join()	.joining(delimiter, prefix, suffix)
.FirstOrDefault(..., 5)	.map(...).orElse(5)
.Single(...)	.filter(...).orElseThrow()

Example?

<https://habr.com/ru/company/otus/blog/658999/>